CprE 5500 RPC Project Design Document

This project set out to create a restaurant ordering and management system that utilizes a client and a server application that communicates using CORBA. The client application is responsible for providing the customers of the restaurant along with the manager with an interface to be able to view the restaurant menu along with placing and viewing orders. The server application is responsible for maintaining the current menu and orders that the restaurant has along with providing them out to any clients that request the information. The two services communicate via CORBA which is defined in a centralized interface definition file that is used to generate stubbed procedures for both the client and server to use to communicate with one another. This design document will take a closer look at how the client and server application interact with one another via this CORBA communication protocol.

Interface Definition

This project began by defining the interfaces so that the client and the server would be able to communicate with one another. There are three separate interfaces to deal with the three major functions of the system. The first is a simple menu interface that allows the client to access the current menu of the restaurant. The second is the orders interface which handles the processes of placing orders and customers being able to review their orders. Finally, there is the admin interface which allows the admin to authenticate with the server along with setting up a new menu and reviewing all the orders that the server has taken from all customers. The interface definition file has custom exceptions defined to allow the server to communicate specific issues back to the client. With the interfaces and exceptions defined stubs could be generated for both the client and server to implement.

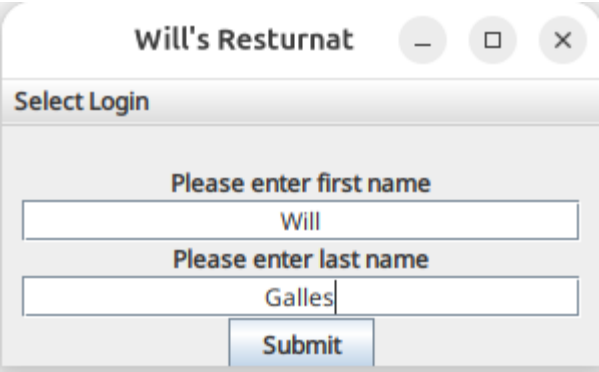
```
interface Menu_Int
{
  attribute Menu menu;
};

interface Order_Int
{
  attribute OrdersList orders;
  boolean placeOrder(in Order order) raises (Empty_Order, Menu_Too_Old, Incorrect_Order_Total, Order_In_Progess);
  Order getActiveOrder(in string userId) raises (No_Active_Order);
  OrdersList getPreviousOrders(in string userId);
};

interface Admin_Int
{
  long getAdminKey(in string username, in string password) raises (Incorrect_Password);
  boolean setMenu(in MenuList menu, in long key) raises (Incorrect_Key);
  OrdersList getAllOrders(in long key) raises (Incorrect_Key);
  OrdersList getAllActiveOrders(in long key) raises (Incorrect_Key);
};
```

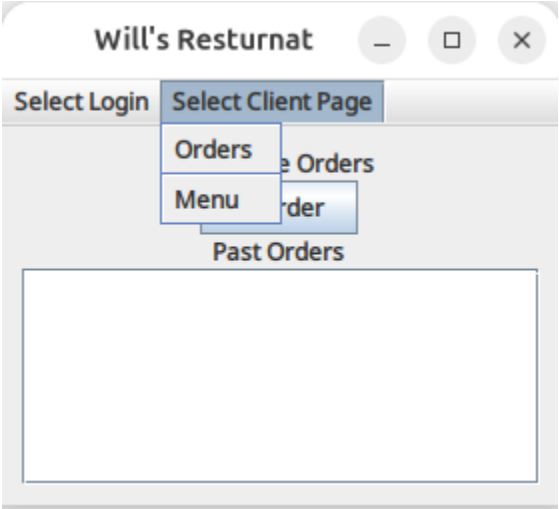
Client Login

The first page that the client is greeted with is the customer login page. This page allows the customer to enter their information into the system. This information will be held locally in the client until there comes a time when the customer needs to request specific information from the server (get previous orders, place a new order). Once the customer submits their information they are forwadred on to the orders page.



Client Page Selection

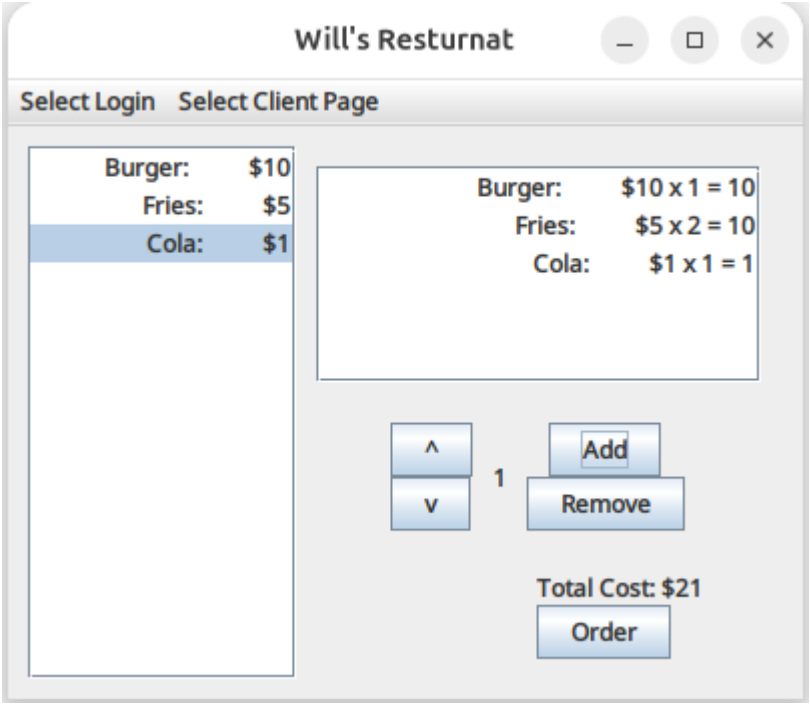
Once the customer has logged in with their information, they have the ability to navigate to two different pages. The orders page will show them their active orders along with any past orders. The menu page will display the latest menu for the restaurant and give the customer the ability to place an order. The page selection menu also allows the user to refresh the page that they are currently viewing by reselecting it from the drop down. This would allow the user to pull in the latest information about their orders and any menu updates.

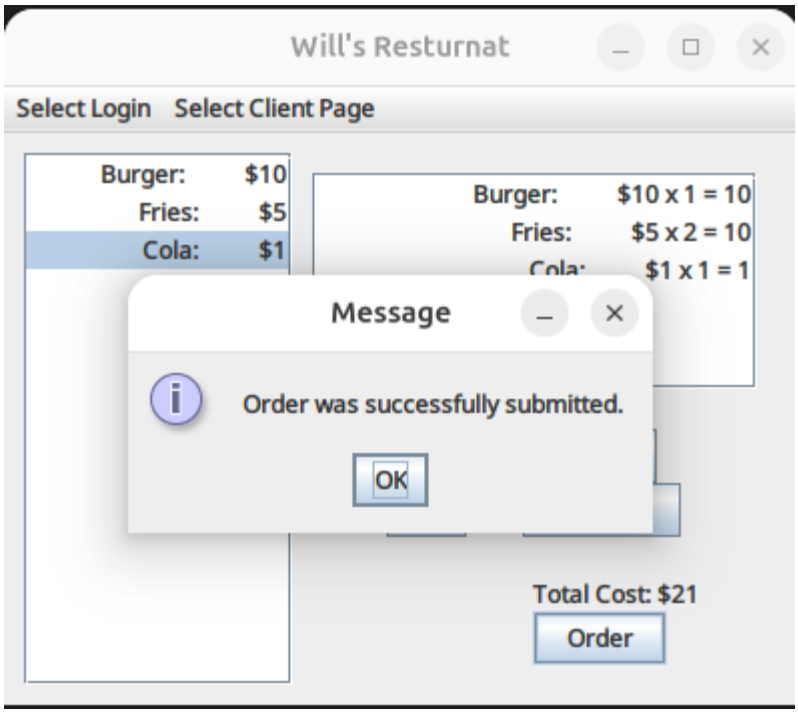


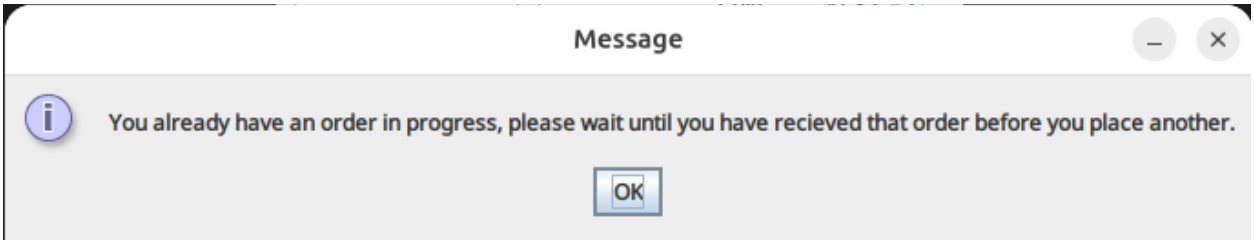
Client Menu Page

When the client menu page is loaded, it reaches out to the server for the latest menu. It does this through with a get method call on the menu interface's "menu" attribute. This object request by the client is fulfilled when the server sends the current menu object back to the client. Once the client has received the menu from the server, it is displayed to the customer. From there, the customer can select what menu items and the

quantity they would like. When the user is satisfied with their order, they can send it back to the server using the placeOrder() method in the order interface. The client will send the new order over to the server; the server will then verify that it is a valid order (correct cost, no ongoing orders). If the order is valid, the server will return an affirmation to the client which is in turn displayed to the customer. However, if there is an issue with the order, the server will throw an exception, and the client will display an informative prompt to the user about what went wrong with their order. If the customer is successful in placing an order, they will be forwarded to the orders page to review their orders.



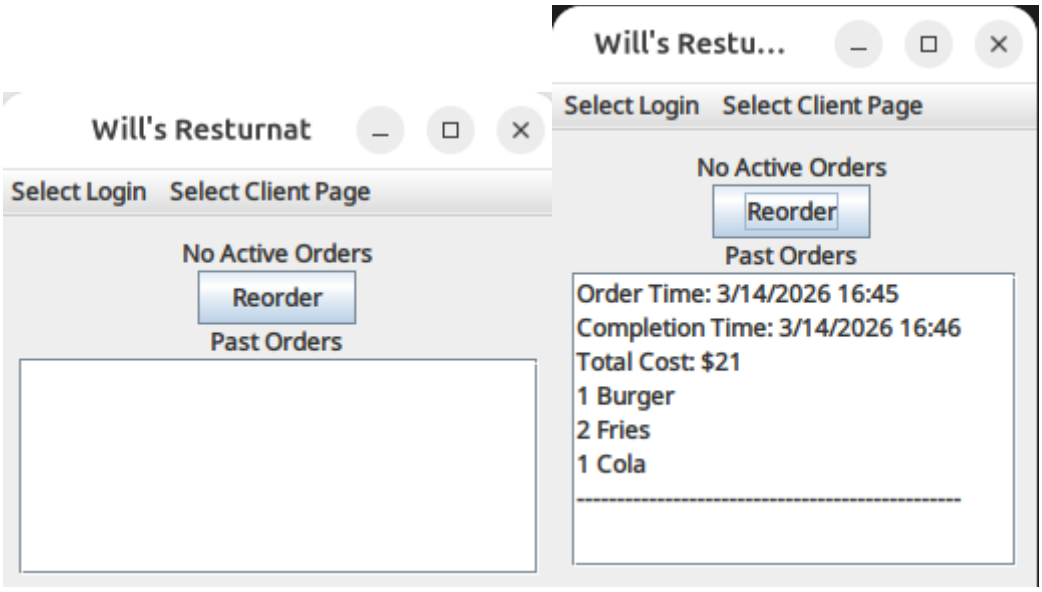


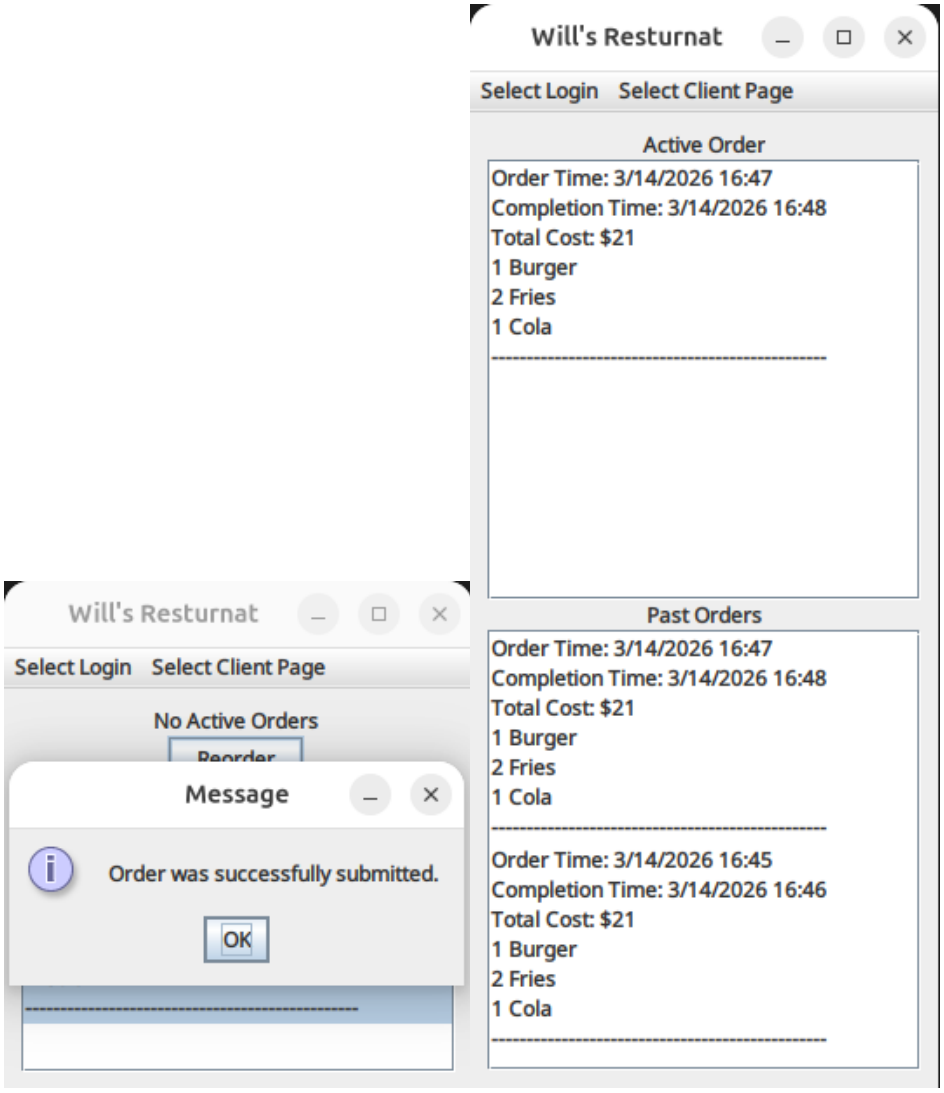


Client Orders Page

When the client orders page is loaded, there are two requests that are sent to the server from the client. The first request is the getActiveOrder procedure from the order interface. This method has the client send over their userId (which was created at the login page) to the server. The server then looks through its list and tries to find out if that user has an active order in progress (orders whose completion time has not yet passed). If the user has an active order, the server will return the order back to the client to display. If there is no active order, the server will throw a new No_Active_Order exception. If there is an active order received back by the client, it will display for the customer to view. However, if there is not an active order in progress, the client will instead show a reorder button to the customer. This reorder button will be used in conjunction with the second server request that is made when the page is initially loaded. The second request to the server is for all the previous requests that this customer has placed. The client again sends their userId with the getPreviousOrders() method to the server. The server will then look through all the

orders and compile a list of orders that were placed by this customer and return it back to the client. The client will then display all the previous orders that this customer has placed. With the previous orders now displaying, the customer can select one and press the reorder button for a quick reorder option. This will then call the placeOrder() method again and pass the same order back to the server. The server will then handle the new order in the same way as the intial order and return the same status back to the client.



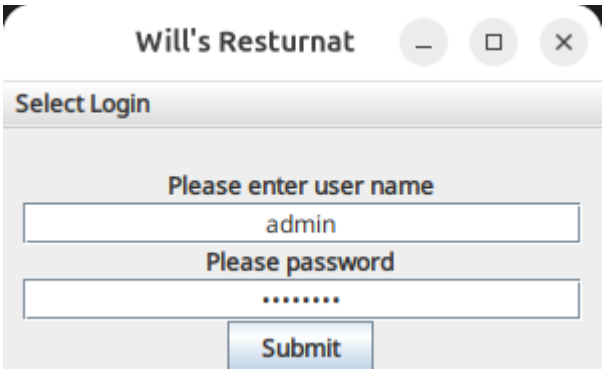


Login Selection

At any time both customers and admins can return to the login pages using the select login menu. The user is also logged out when the select a new login page to transition to.

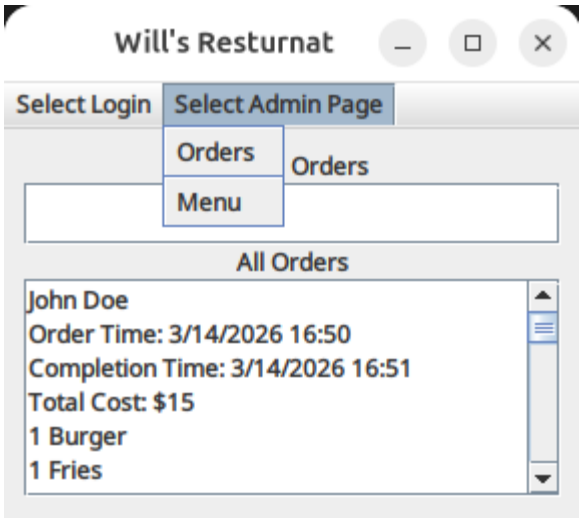
Admin Login

The admin login page is slightly different from the customer login page. This page instead takes the admin username and their password. Once the admin has entered their information, they can submit it to the server for verification. This is done with the getAdminKey() method in the admin interface. The client sends the inputted username and password; the server then compares them with the stored secret values for the username and password. If they are incorrect, the server throws a new Incorrect_Password exception to let the client know that they do not have the correct credentials. If the credentials are correct, an authentication key is sent back to the client to use again in later admin requests. If the client receives a valid authentication key it stores it off for later use and transitions to the admin orders page.



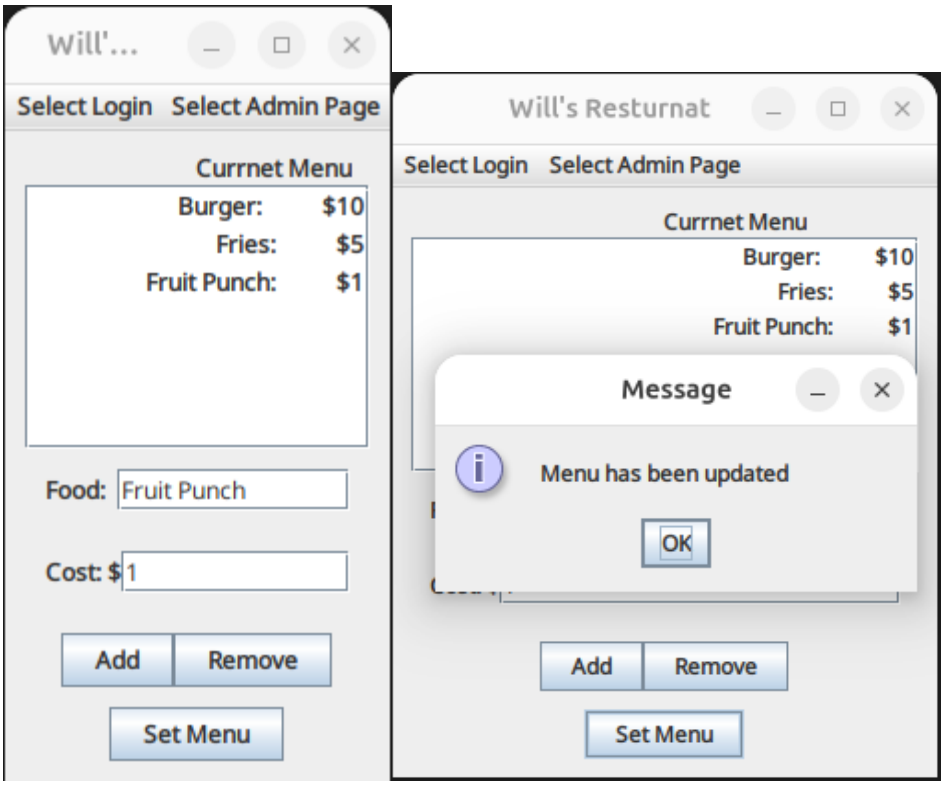
Admin Page Selection

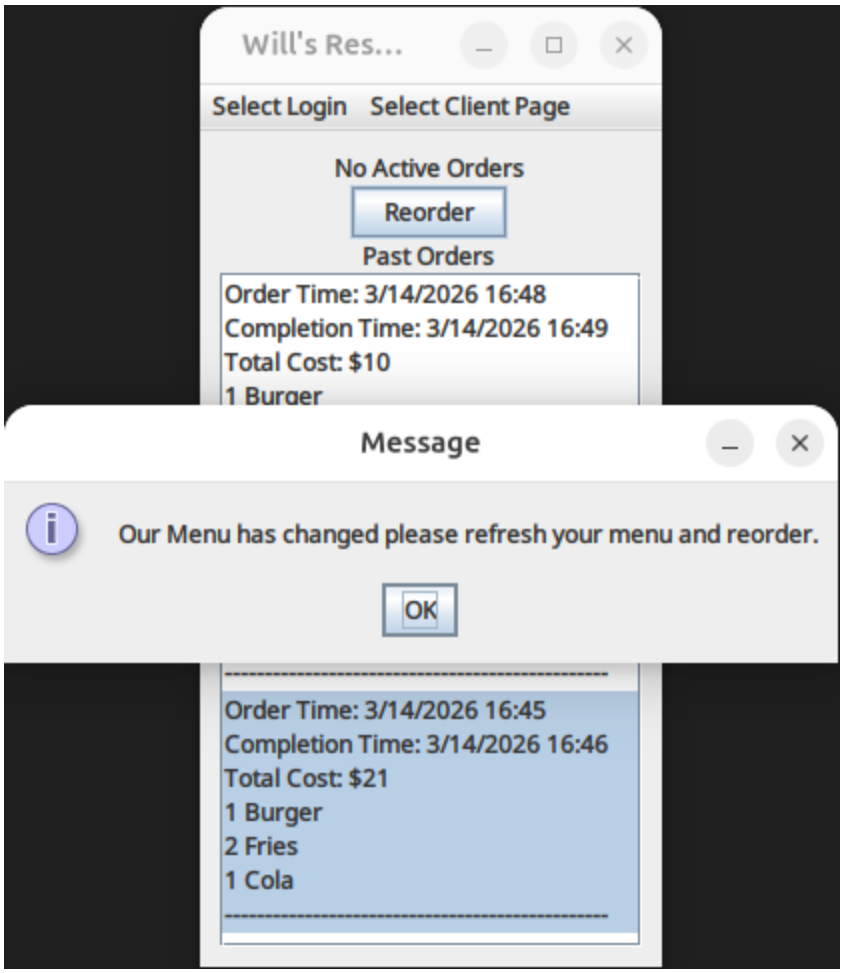
Once the admin has logged in with their information, they have the ability to navigate to two different pages. The orders page will show them all the active orders along with all the previous orders. The menu page will display the latest menu for the restaurant and give the admin the ability to modify and set a new menu. The page selection menu also allows the user to refresh the page that they are currently viewing by reselecting it from the drop down. This would allow the user to pull in the latest order and menu information.



Admin Menu Page

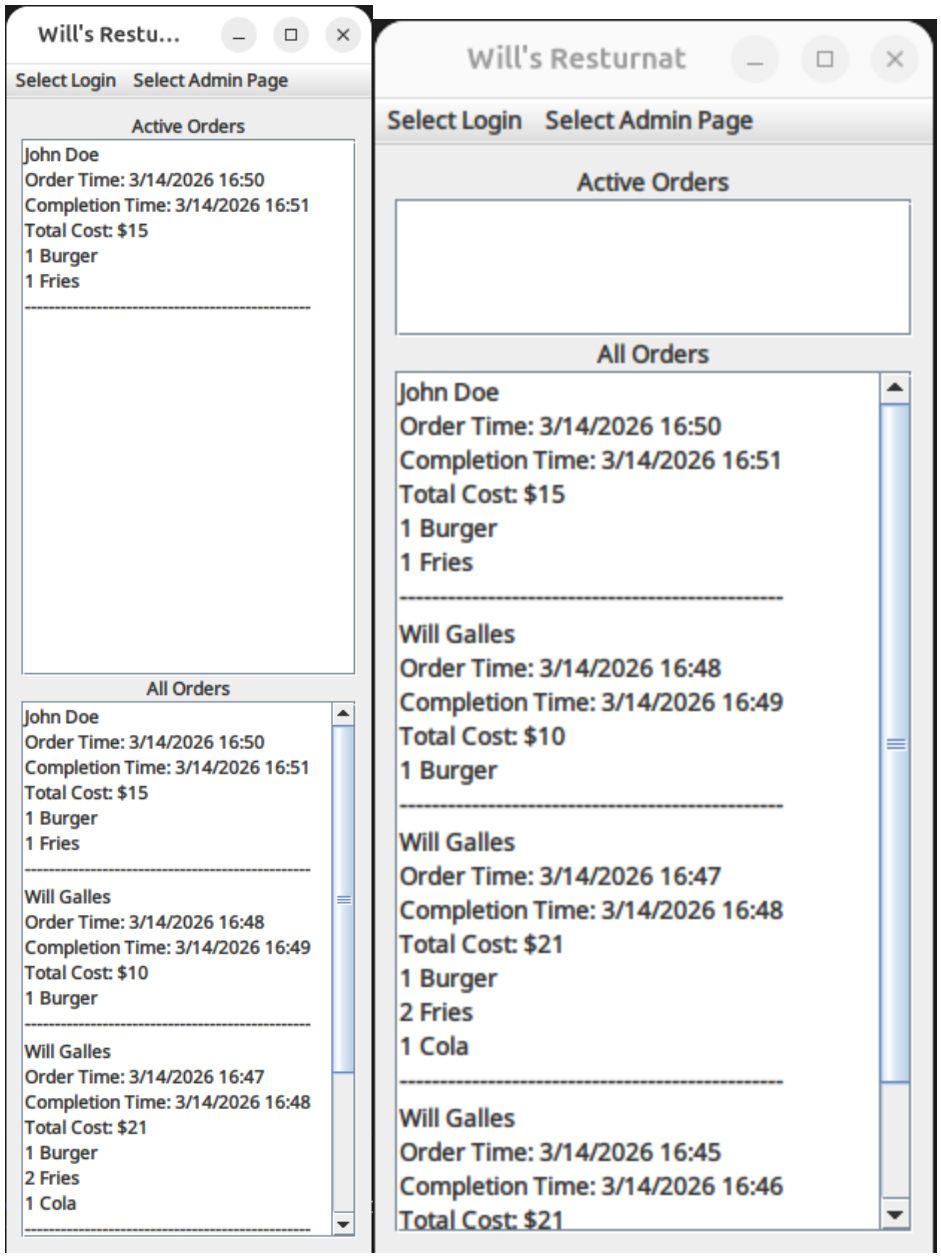
When the admin menu page is loaded, it reaches out to the server for the latest menu. It does this again like the client with an accessor call to the server to get the latest menu. Once the menu has been, the client displays it and allows the admin to modify it. The admin has the ability to add new items and set their price along with removing items that are currently on the menu. Once the admin is satisfied with the new menu, they can send it back to the server to set it as the active menu with the set menu button. Once this button is pressed. This is done with the setMenu() method in the admin interface. The client sends its authentication key along with the new menu over to the server. The server first verifies that the admin has the correct authentication key. If it is not correct, the server will throw a new Incorrect_Key exception. The client will then display this issue and encourage the admin to reauthenticate. If the key is correct, the server will then set its internal menu to the new one submitted by the admin and return a success indication to the client. This client will then display to the admin that it was a success, and the menu has been updated. Every time that a new menu is set the server will increment the menu version. This allows the server to verify that all orders being placed with customers are using the correct menu. If a customer tries to order with an outdated menu a Menu_To_Old exception will be thrown the client will inform the customer that they need to update their menu.


Will'...
Select Login
Select Admin Page
Currnet Menu
Burger: $10
Fries: $5
Fruit Punch: $1
Food: Fruit Punch
Cost: $1
Add
Remove
Set Menu
Will's Resturnat
Select Login
Select Admin Page
Currnet Menu
Burger: $10
Fries: $5
Fruit Punch: $1
Message
Menu has been updated
OK
Add
Remove
Set Menu



Admin Orders Page

When the admin orders page is loaded, the client will send out two similar requests to the server. Both getAllOrders() and getAllActiveOrders will be sent with the client's authentication key as a parameter. The server will again verify that the authentication key is correct and will throw an exception if it is not correct. If the key is valid, the server will return its whole orders list for the call to get all orders and return a reduced set of active orders for the active request. The client will then display these two lists of orders for the admin to review.



Server Responsibilities

The server is mainly responsible for responding to the requests from the clients by validating their requests and returning them to the desired information. There is not much else the server is doing for this application other than being a data storage device. The majority of the server functionalities were described alongside the individual client actions.

Conclusion

This project had us utilize the CORBA communication protocol to communicate between multiple clients and a singular server to create our restaurant management system. We first defined the common interfaces that both the client and server would use in an interface definition file. We then took that idf and generated stubs that we would then extend with both the client and the server. The client would be an interactable system that would allow customers and admins to manage and interact with menus and orders. The server would be a standalone system that would handle incoming data requests from multiple clients and persist in data while clients would come and go. These three portions would complete our restaurant management project.